

TrendScope

Complete Project Documentation

Pace Techfeast '26

July 2026

Table of Contents

1. Project Overview
2. Project Structure
3. Python Web Application (Flask)
4. Python Desktop Application (Tkinter)
5. Android Native Application (TrendScope)
6. Android Architecture & Components
7. Translation System
8. RSS News Aggregation
9. Setup & Installation
10. API Reference

1. Project Overview

TrendScope is an intelligent, real-time news aggregation platform designed to cut through the noise. It allows users to instantly fetch, filter, and read trending news across 25 unique domains and 85+ sub-topics from a clean, elegant interface. The project includes three distinct interfaces:

- Python Web Dashboard (Flask) - Browser-based interface with dark/light mode, PDF export, and live activity logs
- Python Desktop App (Tkinter) - Native Windows GUI with domain/sub-topic selection and article viewer
- Android Native App (TrendScope) - Full-featured mobile app with 24 categories, emoji icons, favicons, on-device translation for 10 Indian languages, day-based article grouping, and bookmark support

All interfaces share a common RSS-based news aggregation engine powered by Google News RSS feeds and public news sources across 25 domains including AI, Technology, Sports, Finance, Health, Science, Cybersecurity, Gaming, Crypto, Entertainment, World News, and more.

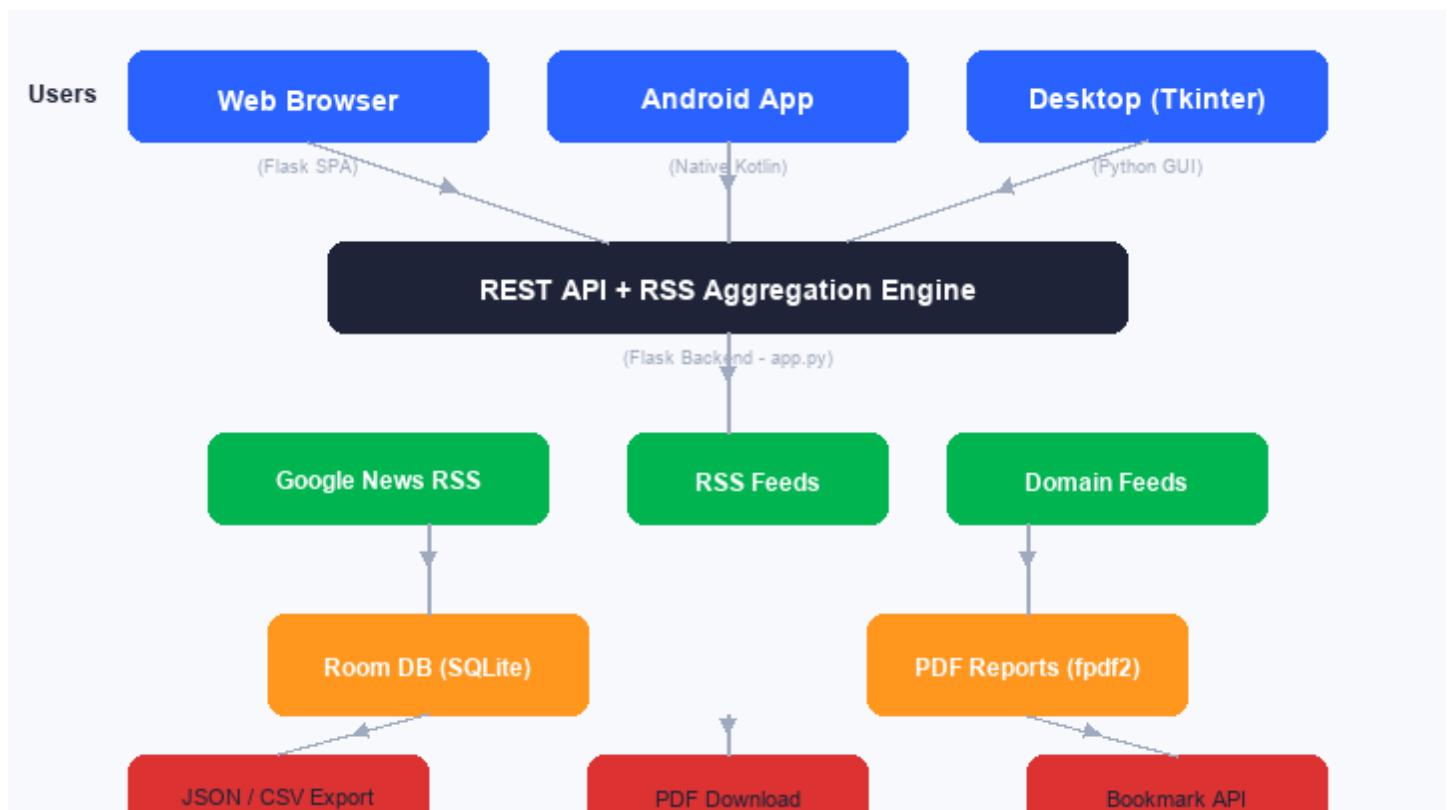


Figure 1: System Architecture - Users, Backend, Storage & Output

2. Project Structure

Directory Tree

```

pace_tech-feast--main/
+-- app.py                # Flask backend - API routes, RSS parsing, PDF gen
+-- main.py               # Tkinter desktop GUI application
+-- launcher.py           # Flask + Cloudflare tunnel launcher
+-- viewer.py             # Article display helper for Tkinter
+-- pdf_opener.py         # HTML-based PDF viewer
+-- clean_layout.py       # Generates templates/index.html
+-- build_data.py         # Generates _data.json (domains + images)
+-- build_html.py         # Full HTML/CSS/JS generator (653 lines)
+-- requirements.txt      # Python dependencies
+-- README.md             # Project documentation
+-- TODO.md              # Task list
+-- templates/
|   +-- index.html        # Single-page web app (generated)
+-- synapse/
|   +-- components/
|   |   +-- get_news.py   # NewsScraper class - RSS fetching + parsing
|   +-- config/
|   |   +-- domain_feeds.py # DOMAIN_RSS_FEEDS - 25 domain RSS URLs
|   +-- pipelines/
|   |   +-- supervisor.py  # run_weekly_pipeline - orchestration
+-- android/
    +-- TrendScope/
        +-- build.gradle.kts
        +-- settings.gradle.kts
        +-- app/
            +-- build.gradle.kts
            +-- release.keystore
            +-- src/main/
                +-- AndroidManifest.xml
                +-- java/com/trendscope/app/
                    |   +-- MainActivity.kt
                    |   +-- ArticleActivity.kt
                    |   +-- ArticleAdapter.kt
                    |   +-- CategoryActivity.kt
                    |   +-- CategoryAdapter.kt
                    |   +-- data/
                    |   |   +-- Article.kt, ArticleDao.kt, AppDatabase.kt
                    |   +-- network/
                    |   |   +-- DomainFeeds.kt, RssParser.kt
                    |   +-- translate/
                    |   |   +-- TranslationManager.kt
                +-- res/
                    +-- drawable/, layout/, menu/, mipmap-*/
                    +-- values/ (10 Indian languages + English)

```

3. Python Web Application (Flask)

The Flask web backend serves as the primary API server for the web dashboard. It provides RESTful endpoints for fetching news, generating PDF reports, managing bookmarks, and exporting data. The frontend is a single-page HTML application generated by `build_html.py` / `clean_layout.py`.

Key Features

- 25 domains with 85+ sub-topics for granular news filtering
- Real-time RSS feed fetching with date-range filtering (1/7/30 days)
- Advanced search combining domain, sub-topic, custom query, and time range
- PDF report generation with `fpdf2`
- Trending words extraction across top news domains
- Bookmark management with CRUD API
- Export to JSON and CSV formats
- Live activity log showing fetch progress and errors
- Dark/Light theme toggle with `localStorage` persistence
- Frequent categories quick-access panel

API Routes

Flask Routes

GET	/	Serves the SPA frontend (templates/index.html)
GET	/api/domains	Returns dict of domains -> [subtopics]
GET	/api/news	Fetch articles: ?domain=&subtopic=&days=&search=
GET	/api/trending	Extract top trending words from recent articles
GET	/api/generate-pdf	Generate PDF report for given filters
GET	/api/latest-pdf	Download the most recently generated PDF
GET	/api/frequent-categories	Returns 10 most-used category shortcuts
GET	/api/export/json	Export articles as JSON download
GET	/api/export/csv	Export articles as CSV download
POST	/api/bookmarks	Save a bookmark (JSON body)
GET	/api/bookmarks	List all bookmarks
DELETE	/api/bookmarks	Remove a bookmark (?url=)

News Fetching Architecture

The `_fetch_articles()` function builds a query from the selected domain and sub-topic, constructs RSS feed URLs (combining domain-specific feeds from `DOMAIN_RSS_FEEDS` with a Google News RSS search URL), and uses the `NewsScraper` class to parse articles. Results are filtered by the cutoff date (1/7/30 days).

4. Python Desktop Application (Tkinter)

The Tkinter-based desktop GUI (main.py) mirrors the web app functionality with a native Windows interface. It provides the same domain/sub-topic browsing, article fetching, and PDF report generation.

TrendScopeUI Class

Desktop Application Architecture

```
+-- TrendScopeUI
+-- __init__()      Window setup (1050x680, dark theme)
+-- build_ui()      Constructs all widgets
|   +-- Header: title, subtitle, theme controls
|   +-- Trending bar: horizontal scroll of word clouds
|   +-- Domain listbox (left panel)
|   +-- Sub-topic listbox (center-top)
|   +-- Date range radio buttons (1/7/30 days)
|   +-- Search entry with placeholder
|   +-- Generate PDF / Open Latest buttons
|   +-- Article viewer text box (rich text)
|   +-- Live activity log (right sidebar)
+-- on_main_select() Populates sub-topics for selected domain
+-- on_sub_select()  Stores selected sub-topic
+-- start_generation() Fetches articles + generates PDF
+-- open_latest_pdf() Opens latest PDF via pdf_opener.py
+-- add_log()        Appends timestamped log messages
```

Color Theme

Color Palette

Background (dark)	#0f172a (Slate 900)
Panel background	#1e293b (Slate 800)
Accent	#8b5cf6 (Violet 500)
Text main	#f1f5f9 (Slate 100)
Text subtle	#94a3b8 (Slate 400)
Success	#22c55e (Green 500)
Danger	#ef4444 (Red 500)

5. Android Native Application (TrendScope)

TrendScope for Android is a fully native Kotlin application targeting API 26+ with a modern Material Design interface. It provides the complete news aggregation experience on mobile with additional native features including on-device translation, domain favicons, emoji category icons, and bookmarks.

Build Configuration

build.gradle.kts Summary

```
Plugins:      Android Application, Kotlin Android, KAPT, Kotlin Serialization
compileSdk:   34
targetSdk:    34
minSdk:       26
versionCode:  3
versionName:  3.0.0
Java:         JDK 17
Signing:      release.keystore (TrendScope2024)
ProGuard:     minifyEnabled = true
View Binding: enabled
```

Dependencies

Key Dependencies

androidx.core:core-ktx:1.12.0	App compat
com.google.android.material:material:1.11.0	Material Design
androidx.room:room-runtime:2.6.1	Room Database
androidx.room:room-compiler:2.6.1 (kapt)	Room annotation processor
com.squareup.okhttp3:okhttp:4.12.0	HTTP client
org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3	
io.coil-kt:coil:2.5.0	Image loading
androidx.lifecycle:lifecycle-*:2.7.0	Lifecycle components

Android App Architecture

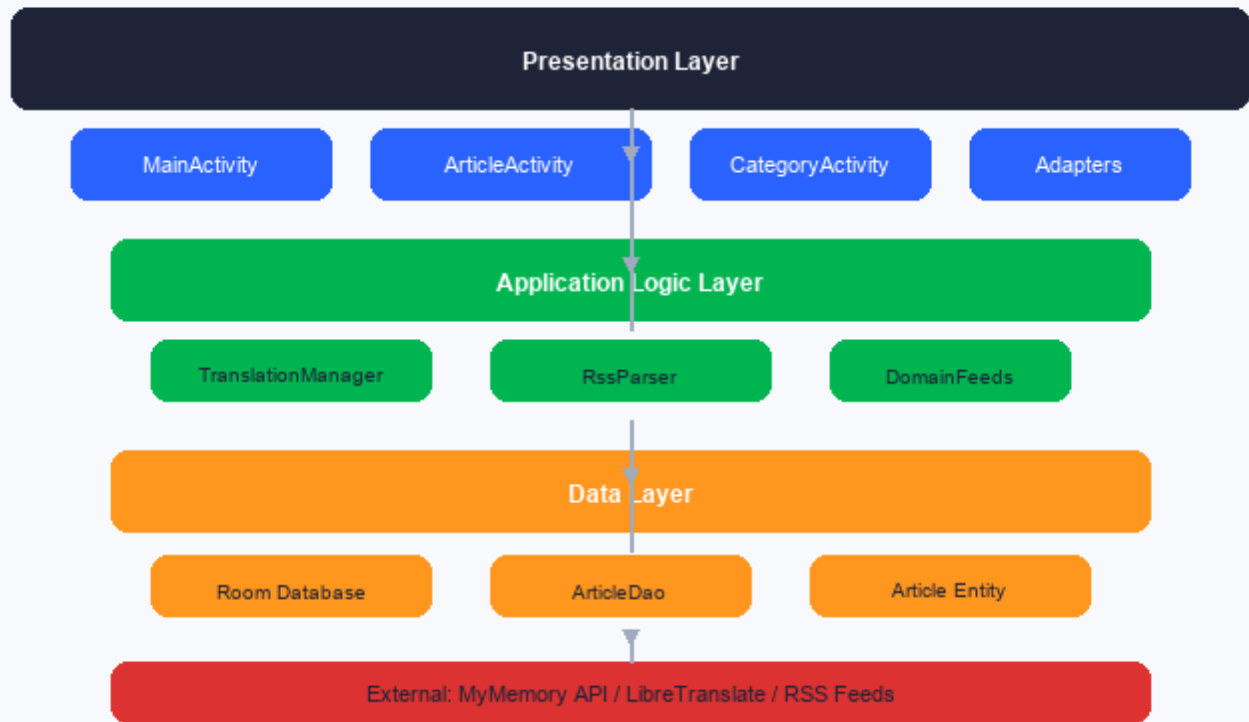


Figure 2: Android App Architecture - Layered Design

6. Android Architecture & Components

Activities (4 screens)

The app uses a standard single-Activity architecture with 3 activities:

MainActivity.kt (289 lines)

The primary activity with three tabs: Categories (grid of 24 emoji-coded categories), All News (chronological article list with day headers), and Bookmarks (saved articles). Includes a toolbar with language selector menu, search functionality, and RSS feed refresh. Manages batch article translation and persistent language preference via SharedPreferences.

CategoryActivity.kt (76 lines)

Opened when a category card is tapped. Displays all articles filtered by the selected category. Supports swipe-to-refresh and passes the selected language for translation display.

ArticleActivity.kt (143 lines)

Full article detail view showing title, source, date, content, and image. Features include bookmark toggle (heart icon), share via Android intent, 'Open Original' button loading the source URL in browser, and translated content display when a non-English language is active.

Data Layer

Article.kt (Room Entity)

Article Entity

```
@Entity(tableName = "articles")
data class Article(
    @PrimaryKey val link: String,
    val title: String,
    val description: String,
    val content: String = "",
    val pubDate: String,
    val imageUrl: String = "",
    val source: String,
    val category: String,
    val isBookmarked: Boolean = false,
    val fetchedAt: Long = System.currentTimeMillis(),
    val pubDateMillis: Long = 0L,
    val translatedTitle: String = "",
    val translatedDescription: String = "",
    val translatedContent: String = "",
    val translateLang: String = ""
)
```

ArticleDao.kt

DAO Operations

Key queries:

- getAllArticles()	-> All articles ordered by date
- getArticlesByCategory()	-> Filtered by category
- getBookmarks()	-> Bookmarked articles only
- searchArticles(query)	-> Full-text search (title, desc, translations)
- insertArticles()	-> Batch insert with REPLACE strategy
- updateTranslation()	-> Update translated text + language code
- toggleBookmark()	-> Toggle bookmark status
- getArticlesNotInLang()	-> Articles needing translation
- getCount()	-> Total article count

AppDatabase.kt*Database*

```
@Database(entities = [Article::class], version = 3)
TrendScope database singleton with destructive migration fallback.
```

View Layer**ArticleAdapter.kt (164 lines)**

RecyclerView adapter that groups articles into sections with day headers (Today, Yesterday, This Week, This Month, Earlier/Monthly). Each article card displays: domain favicon, title (translated when language active), source, date, description preview, and thumbnail image. Uses coroutine-based image loading via raw HTTP connections.

CategoryAdapter.kt (48 lines)

Grid adapter (2 columns) displaying 24 category cards. Each card shows the category emoji and name with a randomly colored background from a curated palette.

7. Translation System

The Android app provides cloud-based translation across 10 Indian languages. The system uses a primary API with automatic fallback for reliability.

Supported Languages

10 Indian Languages

Code	Language	Native Name	Type
hi	Hindi	Hindi	Full
bn	Bengali	Bangla	Full
te	Telugu	Telugu	Full
ta	Tamil	Tamil	Full
mr	Marathi	Marathi	Full
gu	Gujarati	Gujarati	Full
kn	Kannada	Kannada	Full
ml	Malayalam	Malayalam	Full
pa	Punjabi	Punjabi	Full
ur	Urdu	Urdu	Full

Architecture

TranslationManager is a singleton object that provides a suspend function for translation. It uses a two-tier API strategy:

- Primary: MyMemory API (api.mymemory.translated.net) - generous free tier, no API key required
- Fallback: LibreTranslate (libretranslate.com/translate) - open-source, used when MyMemory fails

Translation Flow: English to Indian Languages

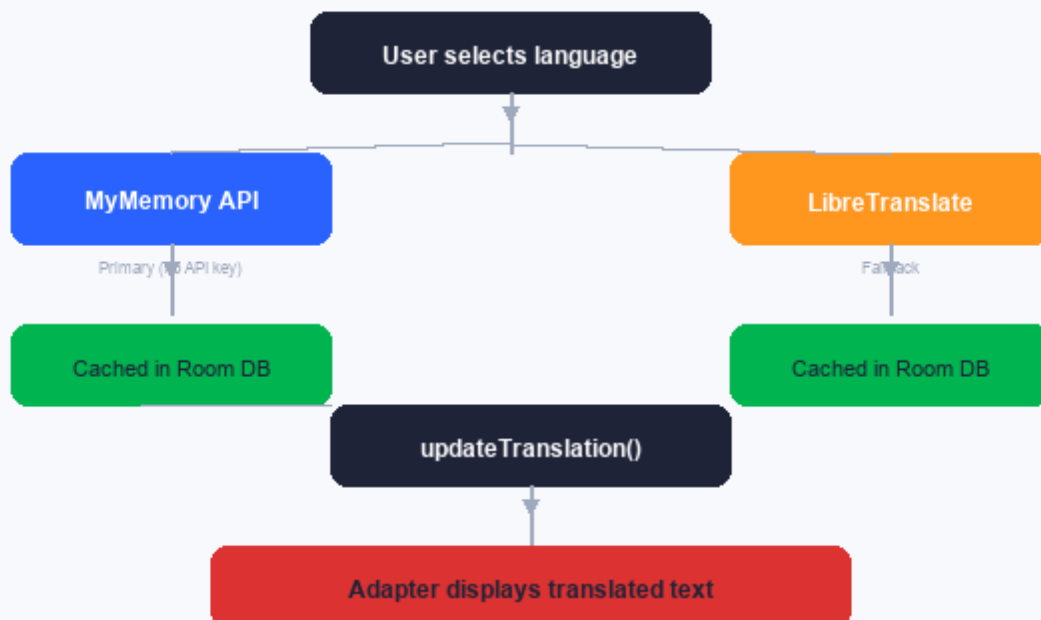


Figure 3: Translation Pipeline - API Call Flow

Translation Flow

Sequential Translation Pipeline

```
User selects language
-> setLanguage(code) called
-> Switch to All News tab
-> translateAllArticles(targetLang) launched
-> getArticlesNotInLang() from Room
-> For each article (sequentially, 600ms gap):
    -> translate(title) via MyMemory -> LibreTranslate fallback
    -> 400ms delay
    -> translate(description)
    -> 400ms delay
    -> translate(content)
    -> updateTranslation() in Room DB
-> refreshCurrentView() loads translated articles
-> New ArticleAdapter displays translated text
```

UI String Resources

All UI strings (tabs, buttons, status messages) are provided in all 10 Indian languages plus English. The app uses Android's locale-based resource resolution with values-`{lang}`/strings.xml files.

21 String Resources per Language

```
Strings include: app_name, tab_categories, tab_all_news, tab_bookmarks,
search_hint, today, yesterday, this_week, this_month, earlier,
open_original, no_bookmarks, bookmarked, removed_bookmark,
select_language, language, status_starting, status_fetching,
status_translating, status_translating_count, translation_complete,
translation_partial, loading_articles
```

8. RSS News Aggregation

24 Categories

Category Index with Emoji Icons

Category	Icon
Artificial Intelligence	AI
Technology	TC
Sports	SP
Finance	FN
Health & Fitness	HF
Education & Careers	ED
Entertainment	EN
Business & Startups	BS
World News	WN
Science & Space	SS
Cybersecurity	CS
Gaming	GM
Mobile & Gadgets	MG
Cryptocurrency	CR
Environment	EV
Travel & Tourism	TR
Food & Lifestyle	FL
Automobiles	AU
War & Conflicts	WC
Social Media	SM
Fashion & Beauty	FB
Politics & Government	PG
Real Estate & Property	RE
Music	MU

RssParser.kt (186 lines)

The RSS parser uses Android's XmlPullParser to parse RSS 2.0 and Atom feeds from multiple sources. Key features:

- Parses title, link, description, pubDate, and image from <enclosure>, <media:content>, or tags
- Date formatting from multiple RSS date formats to 'MMM dd, yyyy HH:mm' display format
- pubDateMillis computed for day-based grouping (Today/Yesterday/This Week/This Month)
- Batch fetching across all 24 categories with per-article callback
- HTML tag stripping from description text
- Per-source RSS feed URLs defined in DomainFeeds.FEEDS map

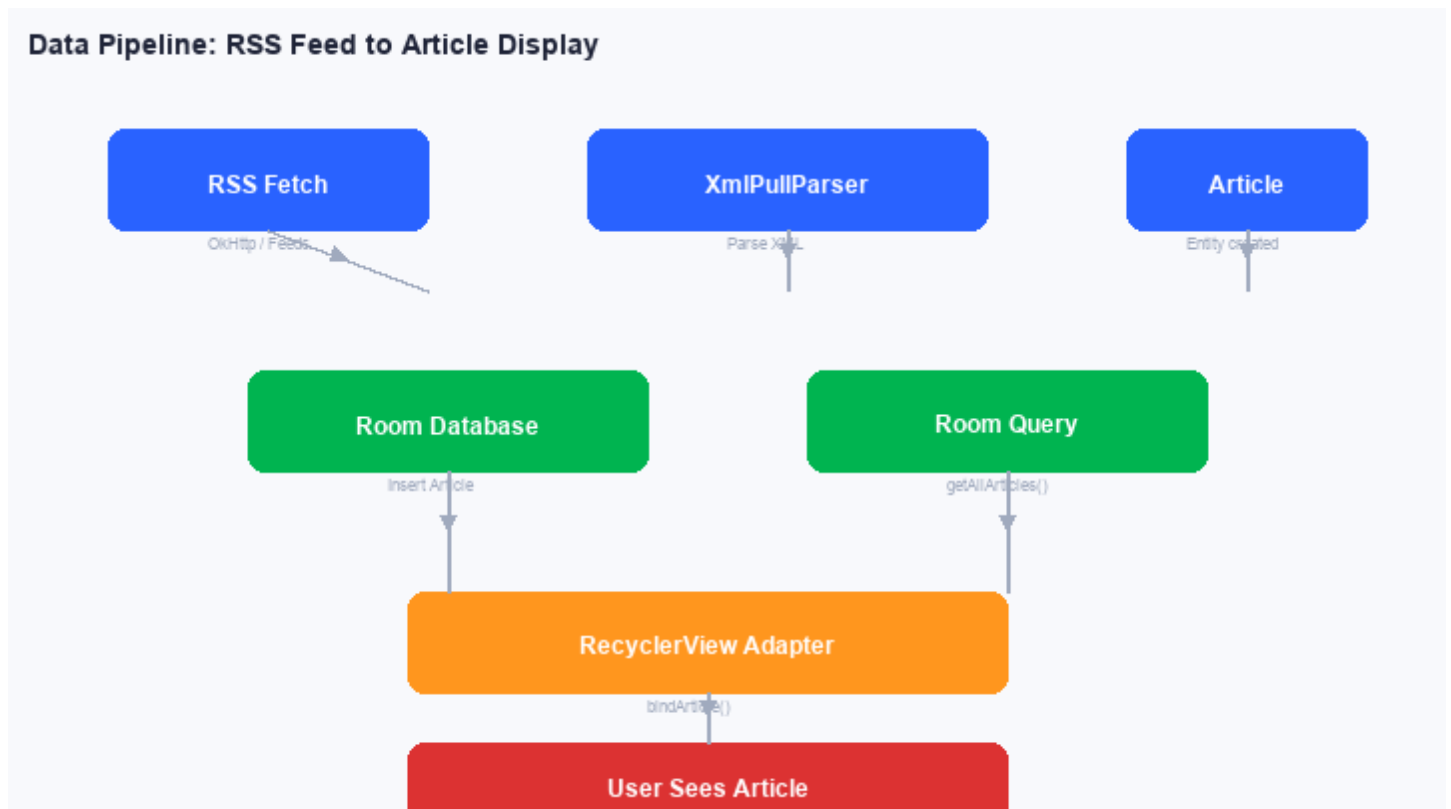


Figure 4: Data Pipeline - RSS Feed to Article Display

Domain Favicons

Each article in the list displays a small favicon next to the source name, loaded from Google's favicon service: <https://www.google.com/s2/favicons?domain=DOMAIN&sz=64>. The `DomainFeeds` object maps source names to domains via `getFaviconUrl()`.

9. Setup & Installation

Python Web App

Python Setup

```
# Prerequisites: Python 3.8+
python -m venv .venv
.venv\Scripts\Activate # Windows
pip install -r requirements.txt
python clean_layout.py # Generate frontend
python app.py # Run -> http://localhost:5000
```

Android App

Android Build

```
# Prerequisites: JDK 17, Android SDK 34
cd android/TrendScope
gradlew assembleRelease
# Output: app/build/outputs/apk/release/app-release.apk
# Signing: release.keystore (TrendScope2024)
# APK Size: ~2.4 MB
```

Build Outputs

Artifacts

```
Python Web App: http://localhost:5000
Python Desktop: python main.py (Tkinter GUI)
Android APK: android/TrendScope/app/build/outputs/apk/release/app-release.apk
APK Size: 2.4 MB (minified + ProGuard)
PDF Reports: data/*.pdf (generated via web UI)
```

10. API Reference

Flask API Endpoints

Full API Reference

```
GET /api/domains
  Returns: {domain_name: [subtopic1, subtopic2, ...]}

GET /api/news?domain=&subtopic=&days=&search=
  Returns: [{title, author, url, date, content, image_url}]

GET /api/trending?domain=
  Returns: [{word, count}] - top 25 trending words

GET /api/generate-pdf?domain=&subtopic=&days=&search=
  Returns: {status, count}

GET /api/latest-pdf
  Returns: PDF file download

GET /api/frequent-categories
  Returns: [{domain, subtopic, label}]

GET /api/bookmarks          List all bookmarks
POST /api/bookmarks         Save bookmark (JSON body)
DELETE /api/bookmarks?url=  Remove bookmark

GET /api/export/json        Export as JSON file
GET /api/export/csv         Export as CSV file
```

Article JSON Schema

Article Format

```
{
  "title":    "Article Title",
  "author":   "Source Name",
  "url":      "https://example.com/article",
  "date":     "2026-07-07",
  "content":  "Full article body text...",
  "image_url": "https://example.com/image.jpg"
}
```

End of Documentation

Generated July 2026 | TrendScope v3.0.0 | Pace Techfeast '26